

MINI PROJECT

on

“AI-DRIVEN RESUME PARSER AND JOB MATCHING SYSTEM”

MASTER OF COMPUTER APPLICATIONS

of

Bengaluru City University



By

PAVAN KARTHIK (P18AJ23S126003)

MUHAMMED MIDLAJ (P18AJ23S126004)

Under the Guidance of

Mrs. Farzeen Basith

Assistant Professor



Acharya Institute of Graduate Studies

Acharya Dr. Sarvepalli Radhakrishnan Road, Acharya P.O.,

Bengaluru- 560107

2024-2025

CERTIFICATE

This is to certify that PAVAN KARTHIK (P18AJ23S126003) and MUHAMMED MIDLAJ (P18AJ23S126004) has completed the 3rd semester mini project work entitled “**AI-DRIVEN RESUME PARSER AND JOB MATCHING SYSTEM**” is a result of the Bonafide work carried out during the academic year 2024-2025 under our supervision.

Signature of Internal Guide

Mrs. Farzeen Basith
Professor, Dept. of MCA
Acharya Institute of Graduate Studies

Signature of HOD

Dr. Vibinchander Selvaraj
HoD-Associate Professor, Dept. of MCA
Acharya Institute of Graduate Studies

Sign of Examiner 1

Sign of Examiner 2

DECLARATION

We PAVAN KARTHIK (P18AJ23S126003) and MUHAMMED MIDLAJ (P18AJ23S126004), student of 3rd semester MCA, **Acharya Institute of Graduate Studies** by the **Bengaluru City University** during the academic year **2024–25**, hereby declare that the project entitled “**AI-DRIVEN RESUME PARSER AND JOB MATCHING SYSTEM**” is carried out and completed successfully by us, and is our original work.

PAVAN KARTHIK
(P18AJ23S126003)

MUHAMMED MIDLAJ
(P18AJ23S126004)

ACKNOWLEDGEMENT

Dreams never turn into reality unless a lot of effort and hard work is put into it and no effort bears fruit in the absence of support and guidance. It takes a lot of effort to work through this goal and having something to guide me is always a blessing.

I would like to take this opportunity to thank a few who were closely involved in completing and executing this project. At the outset, I thank God almighty for making my endeavor a success, I would like to express my sincere thanks to the Management of Acharya Institute of Graduate Studies for providing excellent infrastructure and other faculties, which enabled me to sharpen my skills. I would like to express a deep sense of gratitude to **Dr. Gurunath Rao Vaidya**, Principal for having laid tracks that lead me to a bright future.

I sincerely thank **Dr. Vibinchander Selvaraj**, Head of the Department, for his support in providing the necessary faculty, resources, and guidance that enabled me to successfully complete this mini project.

I express my sincere gratitude to the Project Guide **Mrs. Farzeen Basith** for her constant support and valuable suggestions without which the successful completion of this project would not have been possible.

I express my immense indebtedness to all the teachers and staff of the **Department of MCA** for their cooperation and support. Finally, I thank all others, especially my classmates and family members who in one way or another helped me in the successful completion of this work.

PAVAN KARTHIK
(P18AJ23S126003)

MUHAMMED MIDLAJ
(P18AJ23S126004)

ABSTRACT

The AI Resume Parser and Job Matching System is a web application designed to simplify resume analysis and job matching. The goal is to enable users to upload resumes, extract key skills, and identify job roles that align with their abilities, while providing administrative access for managing the platform, streamlining the recruitment process.

The project leverages multiple technologies to achieve its objectives. The front end, built with HTML, Tailwind CSS, and JavaScript, offers an intuitive and accessible interface. The back end, powered by Flask, manages server-side operations. Resumes in PDF format are processed using pdfplumber to extract text, identifying skills such as programming languages or data analysis. These skills are grouped into categories like Programming, Web Development, or Data Analysis using predefined rules. A job recommendation system maps skills to roles like Python Developer or Data Analyst, and a career path feature outlines progression from junior to senior positions. Results are displayed in a sortable, searchable table, with an option to download as a PDF using ReportLab. User and admin data, including resumes, are stored in a MySQL database for efficient access. Secure authentication, implemented with password hashing and session management, supports user and admin login and signup.

The system effectively extracts skills, recommends relevant jobs, and supports user profile management, enhancing recruitment efficiency. Administrative access is included for platform oversight, though its implementation is limited in the current version. Skill extraction may miss complex phrases, which could be improved. The website allows resume uploads, profile updates, and detailed result views, including skill categories and career paths, making it practical and engaging.

To conclude, the AI Resume Parser and Job Matching System delivers a robust solution for resume parsing, job matching, and recruitment management. It combines reliable technology with a user-friendly design to support job seekers, recruiters, and administrators. Future enhancements could include refined skill extraction, broader file format support, full admin functionality, or integration with job platforms. This project establishes a strong foundation for tools that optimize recruitment and career planning.

CONTENTS

1. INTRODUCTION	1
2. LITERATURE SURVEY	2
3. REQUIREMENT AND ANALYSIS	
3.1 HARDWARE AND SOFTWARE SPECIFICATIONS	6
4. SYSTEM DESIGN	
4.1. SYSTEM PERSPECTIVE	7
4.2. ARCHITECTURE OVERVIEW	8
4.3. MODULE DESCRIPTION	9
4.4. CONTEXT DIAGRAM	11
4.5. USE CASE DIAGRAM	12
4.6. DATA FLOW DIAGRAM	12
4.7. SEQUENCE DIAGRAM	14
5. IMPLEMENTATION	
5.1 SOURCE CODE	16
5.2 SCREENSHOTS	21
6. SOFTWARE TESTING AND RESULTS	25
7. CONCLUSION	
7.1 CONCLUSION	28
7.2 FUTURE ENHANCEMENT	28
8. REFERENCES	30

1. INTRODUCTION

The digital era has transformed recruitment, with a surge in resume submissions through job boards and portals creating challenges for recruiters and job seekers. Traditional methods, relying on manual reviews or basic keyword searches, often lead to delays, missed talent, and suboptimal job placements. The AI Resume Parser and Job Matching System addresses these issues by offering a web-based platform that automates resume analysis and job matching. It extracts skills, categorizes them, recommends tailored job roles, and provides administrative oversight, enhancing recruitment efficiency and ensuring precise skill-job alignment. This project aligns with the growing trend of automation in human resources, akin to intelligent matching on platforms like LinkedIn, aiming to revolutionize resume processing and opportunity alignment.

The motivation for this project stems from the inefficiencies of conventional recruitment methods, which are labour intensive and often fail to fully capture a candidate's skill set or match them to suitable roles, leading to frustration and compromised hiring quality. The system leverages modern technologies: pdfplumber extracts text from PDF resumes, rule-based categorization groups skills into areas like Programming or Data Analysis, and a job recommendation engine maps these to roles like Python Developer, also suggesting career paths. Flask manages backend operations, securely storing user and admin data in a MySQL database, while an admin login interface supports platform oversight, though its functionality is currently limited. The system streamlines the process, reducing manual effort and improving job matching accuracy.

The primary objectives of the system are to create an efficient, accessible, and precise platform for resume analysis, job matching, and recruitment management. It processes resumes, extracts and categorizes skills, recommends relevant jobs, and provides administrative tools. Job matching ensures contextual relevance through skill alignment, while user engagement is enhanced via profile management, resume uploads, and downloadable PDF reports using ReportLab. A responsive interface, built with HTML, Tailwind CSS, and JavaScript, ensures accessibility, and secure authentication with password hashing protects data. Designed for continuous improvement, the platform can evolve by refining skill extraction, expanding job mappings, and enhancing admin functionality, empowering job seekers, recruiters, and administrators with a technologically advanced, efficient, and inclusive recruitment experience.

2. LITERATURE SURVEY

Chandak, A. V., et al. (2024) proposed a system that helps improve the hiring process by using machine learning. The system reads resumes, picks out important information, and connects it with suitable job roles. Natural language processing is used to understand the content of resumes. Then, machine learning models match skills and experience with job requirements. This method helps make hiring faster and more accurate by reducing manual work and improving how well candidates are matched with jobs.

The paper titled "Transforming Applicant Tracking Systems with Intelligent Resume Embeddings for Precise Candidate Matching," authored by Ravi Varma Kumar Bevara, was published in 2024. This research introduces Resume2Vec, a method that utilizes intelligent resume embeddings to enhance candidate matching in applicant tracking systems. The approach emphasizes diversity across professional sectors and relevance to real-world recruitment scenarios. The method aims to improve the efficiency of recruitment processes by providing more accurate and context-aware matching between resumes and job descriptions

Bhoir, N., Jakate, M., Lavangare, S., Das, A., & Kolhe (2023) introduced a novel resume parsing solution aimed at enhancing the efficiency of automated recruitment processes. They proposed a hybrid approach combining Spacy Transformer BERT and Spacy NLP methodologies to extract relevant data from unstructured resumes. This method significantly improves the extraction of pertinent information, ensuring more accurate parsing and better alignment between resumes and job requirements

Overview of Existing System

The existing system for resume parsing and job recommendation combines several machine learning techniques to improve the recruitment process. It starts with collecting resumes, which are then pre-processed to ensure they follow a standard format. Key details such as skills, job experience, education, and other important information are extracted from the resumes. This data is analysed using a combination of traditional machine learning and modern deep learning models, such as BERT, to accurately identify and extract relevant information from unstructured resumes.

job positions. This approach takes into account various industries and ensures that the recommendations are relevant to real-world recruitment needs. The matching process is based on the qualifications of candidates and the job requirements.

Performance metrics like precision, recall, and processing time are used to evaluate the effectiveness of the system. It is continuously improved by incorporating feedback, which allows the system to adapt to changing job market trends, new skills, and evolving resume formats. By integrating these technologies, the system aims to make recruitment faster and more accurate, ultimately improving the hiring process.

Limitation of Existing System

- **Computational Complexity:** The resume parser and job recommendation systems utilizing machine learning techniques often require significant computational resources. Processing large volumes of resumes and job descriptions can lead to delays in both training and prediction phases, especially when utilizing complex models like BERT or hybrid NLP approaches.
- **Dependence on Data Quality:** The performance of the resume parsing system heavily depends on the quality and consistency of the data provided. Poorly formatted resumes or incomplete job descriptions can reduce the accuracy of data extraction and result in less relevant recommendations, as these systems rely on accurately identifying key skills, qualifications, and experience.
- **Overfitting and Generalization Issues:** Machine learning models, particularly when using complex embeddings and deep learning, can suffer from overfitting if trained on limited or biased datasets. This means the model might perform well on specific datasets but fail to generalize effectively to new, unseen resumes or job descriptions.
- **Limited Adaptability to Diverse Formats:** Resumes come in varied formats and styles. While advanced parsing models like Resume2Vec aim to address this challenge, there remains a risk that the system might not fully adapt to new or unconventional resume formats, leading to incomplete or inaccurate data extraction.
- **Difficulty Handling Imbalanced Data:** In real-world recruitment, certain job roles or candidate profiles might be underrepresented, leading to an imbalance in the dataset. This

can bias the system toward recommending candidates for popular roles while neglecting less common but equally important job types, thereby impacting the fairness and inclusivity of the recommendations.

- **Interpretability of Recommendations:** Machine learning models, especially those using deep learning embeddings, often operate as "black-box" systems, making it difficult for users to understand how decisions are made. This lack of transparency can undermine trust in the system, especially when users are unsure how the model selects or recommends candidates for specific job roles.
- **High Resource Consumption:** Models that rely on techniques like BERT and advanced NLP require considerable memory and processing power, particularly when analyzing large datasets. This high resource consumption can strain system infrastructure, especially in environments with limited computational capabilities or large-scale recruitment tasks.

Proposed System

The AI Resume Parser and Job Matching System automates resume analysis and job matching to enhance recruitment efficiency. A MySQL database stores user profiles, including names, emails, resume files, and account types (user or admin). Users interact via a web interface (HTML, Tailwind CSS, JavaScript) to sign up, log in, or access a partially implemented admin login, uploading up to 15 PDF resumes securely.

The system uses pdfplumber to extract text from resumes, matching a predefined skills list (e.g., Python, Java, SQL) using regular expressions. Skills are categorized into groups like Programming or Web Development, and a job recommendation engine maps them to roles such as Python Developer or Web Developer, also suggesting career paths (junior to senior). Results, including candidate names, skills, categorized skills, jobs, and career paths, are displayed in a sortable table, with a PDF download option via ReportLab. Authentication ensures data security through password hashing and session management.

The system's accuracy in skill extraction, categorization, and job matching is evaluated using sample resumes. Planned improvements include better skill extraction for complex phrases, support for additional file formats (e.g., DOCX), and full admin functionality. Designed for job seekers and recruiters, the system supports real-world use with potential job board integration and continuous refinement based on feedback.

Advantages of proposed System

Efficient Resume Processing: Automatically extracts skills from PDF resumes using pdfplumber, reducing the time and effort required for manual resume screening.

Accurate Skill Categorization: Organizes extracted skills into meaningful categories like Programming, Web Development, or Data Analysis, making it easier to understand candidate capabilities.

Tailored Job Recommendations: Maps skills to relevant job roles, such as Python Developer or Data Analyst, ensuring precise and contextually appropriate job matches.

Career Path Guidance: Provides a structured career progression plan, suggesting junior, mid-level, and senior roles based on identified skills, aiding long-term career planning.

User-Friendly Interface: Built with HTML, Tailwind CSS, and JavaScript, the platform offers an intuitive and responsive design, accessible to job seekers, recruiters, and administrators.

Secure Authentication: Implements password hashing and session management for safe user and admin login, protecting sensitive data and ensuring secure access.

Flexible Result Management: Allows users to sort, search, and download analysis results as PDF reports using ReportLab, enhancing usability and portability.

Scalable Database Storage: Stores user profiles, resumes, and analysis results in a MySQL database, enabling efficient data retrieval and management.

Administrative Oversight: Includes an admin login interface (though partially implemented), providing potential for platform management and user monitoring.

Streamlined Recruitment: Simplifies the hiring process by automating skill extraction and job matching, benefiting both recruiters job seekers exploring opportunities.

3. REQUIREMENT AND ANALYSIS

3.1 Hardware Requirements

- Intel i3 Processor or equivalent
- 8 GB RAM (to support Flask server, database queries, and PDF processing)
- Hard drive or SSD (for storing resumes, database, and application files)
- GPU: Graphical Processing unit

3.2 Software Requirements

Web Framework: Flask for backend server development and request handling

Data Processing Tools: pandas for organizing resume data, pdfplumber for PDF text extraction

Database: MySQL for storing user profiles, resumes, and authentication data

Frontend Technologies: HTML, Tailwind CSS, JavaScript for responsive and interactive user interface

PDF Generation: ReportLab for generating downloadable PDF reports of resume analysis

Integrated Development Environment (IDE): Visual Studio Code for writing and debugging code

Programming Language: Python v3.9 or later (for backend logic and resume processing)

Operating System: Windows, macOS, or Linux (compatible with Flask and MySQL)

Security Libraries: Werkzeug for password hashing and session management

Additional Dependencies: Listed in requirements.txt (e.g., mysql-connector-python, cryptography, Jinja2)

4. SYSTEM DESIGN

System design is a pivotal phase in the software development life cycle (SDLC), where the architecture and data flow of the system are meticulously planned. For the Personalized News Aggregator, the design process involves defining a client-server architecture, identifying essential modules, detailing user interactions, and establishing how data circulates between components. This section encompasses both high-level overviews and detailed design perspectives to ensure a robust and user-centric news delivery system.

4.1 System Perspective

The System Perspective of the AI Resume Parser and Job Matching System adopts a client-server model, designed as a full-stack web application with a modular, scalable architecture and separation of concerns to deliver automated resume analysis and job matching while enhancing user and admin efficiency. The Frontend Layer, built with HTML, Tailwind CSS, and JavaScript, renders the user interface, manages interactions such as resume uploads and profile updates, and displays results in sortable, searchable tables with PDF download options. The Backend Layer, developed with Flask, handles API logic, secure authentication using Werkzeug for password hashing and session management, and core business logic for resume processing. It interfaces with a MySQL database to store and retrieve user profiles, resume files, and analysis results. The Database Layer maintains user and admin data, including names, emails, phone numbers, resume files, and skill analysis outputs, ensuring efficient data access for critical processes.

Key processes include Resume Acquisition, which accepts PDF resumes (up to 15 per upload) and processes them using pdfplumber to extract text. Skill Categorization employs rule-based matching with regular expressions to identify and group skills into categories like Programming, Web Development, or Data Analysis. Job Matching maps extracted skills to roles such as Python Developer or Data Analyst using a predefined skill-to-job mapping, while a career path feature suggests junior, mid-level, and senior roles. Output Visualization presents results in a dynamic table, with pandas organizing data and ReportLab generating downloadable PDF reports. The system is scalable, supporting additional file formats or skill categories by updating backend logic and MySQL schemas. It assumes reliable PDF inputs and sufficient system resources, initially constrained to processing 15 resumes per upload, with future optimization planned for handling complex skill phrases, integrating job boards, and fully implementing admin functionality

4.2 Architectural Overview

- **Client-Server Model:**

Client: Frontend built with HTML, Tailwind CSS, and JavaScript handles user interactions, resume uploads, profile management, and result display.

Server: Flask-based backend manages API endpoints, resume processing, database interactions, and PDF generation.

- **Data Flow:**

- Users upload PDF resumes through the frontend which are sent to the backend via form submissions.
- The backend processes resumes using pdfplumber to extract text and identifies skills based on a predefined list. Skills are categorized (e.g., Programming, Web Development) and mapped to job roles (e.g., Python Developer, Data Analyst).
- Extracted data (name, skills, job recommendations, career paths) is structured using pandas and stored in a MySQL database.
- Results are rendered in a sortable, searchable table on the frontend (results.html), with an option to download as a PDF using ReportLab.
- User authentication is handled through secure session management and password hashing, with user data stored in MySQL.
- Administrative access is partially implemented for potential platform oversight.

- **External Interactions:**

Users: Upload resumes, manage profiles, view categorized skills, job recommendations, and career paths, and download PDF reports. Admins (if fully implemented) could manage user data and platform settings.

MySQL Database: Stores and retrieves user profiles, resume files, extracted skills, and authentication details.

External Libraries: pdfplumber processes PDF resumes, pandas structures data, ReportLab generates PDF reports, and Werkzeug secures authentication. No external APIs are used for data fetching.

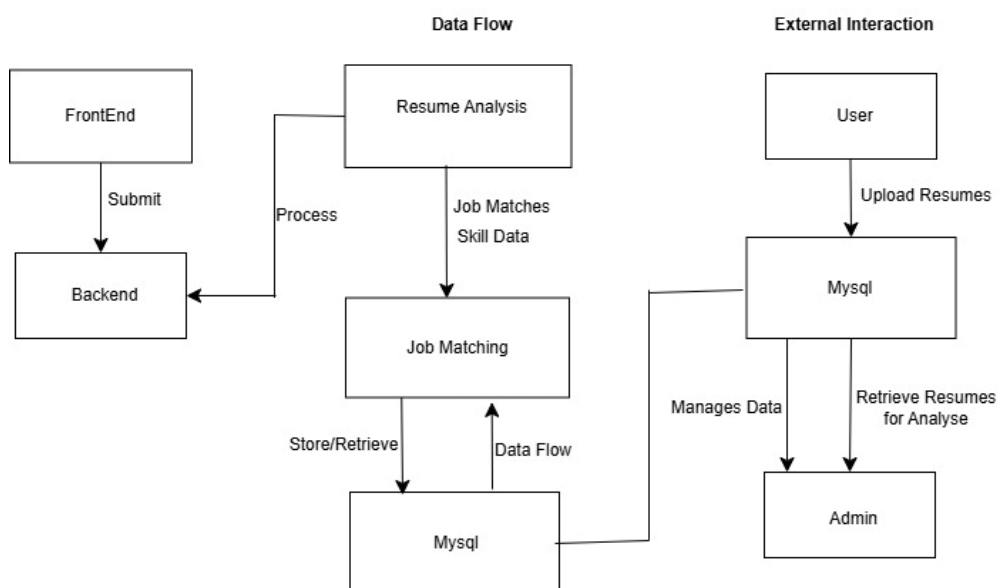


Fig 4.2.1: System Architecture

4.3 Module Description

The AI Resume Parser and Job Matching System comprises five core modules: Web Interface, Server Processing, Resume Analysis, Job Matching, and Authentication. Below is a concise description of each module's purpose, key functionalities, inputs, outputs, and interactions.

4.3.1 Web Interface Module

The Web Interface Module serves as the central hub of the AI Resume Parser and Job Matching System, designed to deliver a clear and accessible platform for users and admins to manage resumes, view analysis results, and perform administrative tasks, ensuring a seamless and efficient experience. This module effectively presents user-friendly pages for registration, sign-in, resume uploads, and result viewing, while enabling admin actions like viewing and downloading resumes, all styled with Tailwind CSS for a responsive design. It handles user interactions such as form submissions for authentication and resume uploads, displays analysis results in a sortable/searchable table (skills, job recommendations, career paths), and provides PDF download functionality powered by ReportLab. The module takes in essential inputs including user/admin credentials (username, password), uploaded PDF resumes, and analysis data (skills, jobs, career paths), transforming them into valuable outputs such as HTTP requests for authentication and uploads, rendered HTML pages with updated results, and downloadable PDF reports. The module interacts dynamically with the Flask Server to process requests and engages users directly through the browser, creating an intuitive and efficient platform for recruitment and career planning.

4.3.2 Backend Server

The Backend Server Module serves as the backbone of the AI Resume Parser and Job Matching System, designed to manage backend operations and coordinate data flow between components, ensuring robust and secure processing of resume data. This module efficiently handles user and admin authentication using password hashing and session management, stores uploaded resumes, processes resume files for skill extraction and analysis, and generates downloadable PDF reports. It takes in essential inputs including user/admin credentials, uploaded PDF resumes, and processing requests from the Web Interface, transforming them into valuable outputs such as stored resume data, processed skill extraction results, categorized skill lists, job recommendations, career path suggestions, and generated PDF files. The module interacts dynamically with the Web Interface to receive requests, collaborates with the Resume Analysis and Job Matching modules for data processing, and engages with the MySQL Database and Authentication module to store and retrieve data, creating a reliable and scalable backend platform for recruitment tasks.

4.3.3 Resume Analysis

The Resume Analysis Module serves as the analytical core of the AI Resume Parser and Job Matching System, designed to extract and categorize skills from uploaded resumes, providing a foundation for accurate job matching. This module effectively utilizes pdfplumber to extract text from PDF resumes, applies regular expressions to identify predefined skills (e.g., Python, Java), and categorizes them into meaningful groups such as Programming, Web Development, or Data Analysis. It takes in essential inputs including uploaded PDF resume files from the Server Processing module, transforming them into valuable outputs such as extracted text and categorized skill lists. The module interacts dynamically with the Server Processing module to receive resumes for analysis and collaborates with the Job Matching module to provide categorized skills, creating a precise and efficient skill identification process for the system.

4.3.4 job Matching Module

The Job Matching Module serves as the decision-making engine of the AI Resume Parser and Job Matching System, designed to recommend job roles and career paths based on analyzed skills, enhancing recruitment and career planning. This module effectively maps categorized skills to relevant job roles (e.g., Python Developer, Data Analyst) using a predefined skill-to-job mapping, and generates career path suggestions outlining progression from junior to senior positions. It takes

in essential inputs including categorized skill lists from the Resume Analysis module, transforming them into valuable outputs such as recommended job roles and career path suggestions. The module interacts dynamically with the Resume Analysis module to receive skill data and collaborates with the Server Processing module to deliver job recommendations for result display, creating an insightful and tailored job matching experience.

4.3.5 Authentication Module

The Authentication Module serves as the security guardian of the AI Resume Parser and Job Matching System, designed to ensure safe access for users and admins, protecting sensitive data and enabling efficient account management. This module effectively manages registration and sign-in processes using password hashing and session handling, validating user and admin credentials securely. It takes in essential inputs including user/admin credentials (username, password), transforming them into valuable outputs such as authentication status and session data. The module interacts dynamically with the Web Interface to receive credentials, collaborates with the Server Processing module to validate access, and engages with the MySQL Database to store and retrieve user/admin data, creating a secure and reliable authentication framework for the system.

4.4 Context Diagram

The Context Diagram provides a simplified, high-level view of a system, showing interactions with external entities like users, APIs, and databases. It depicts data flow into and out of the system, defining boundaries and dependencies without internal process details. Useful in early design, it helps stakeholders grasp system-environment links. This visual tool improves communication, planning, and coordination among developers, designers, and decision-makers

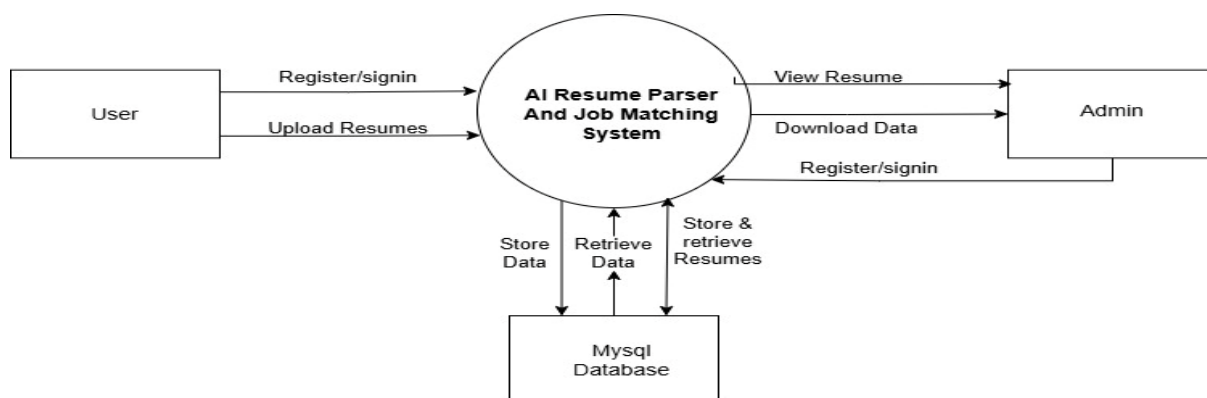


Fig 4.4.1: Context Diagram

4.5 Use Case Diagram

The Use Case Diagram visually depicts a system's functional requirements, highlighting key actors and their interactions with use cases. It illustrates what the system does from the user's view, such as browsing, submitting input, or accessing services. This diagram clarifies behaviour, ensuring features like authentication and content delivery are covered. It serves as a development guide and communication tool, aiding technical teams and non-technical stakeholders

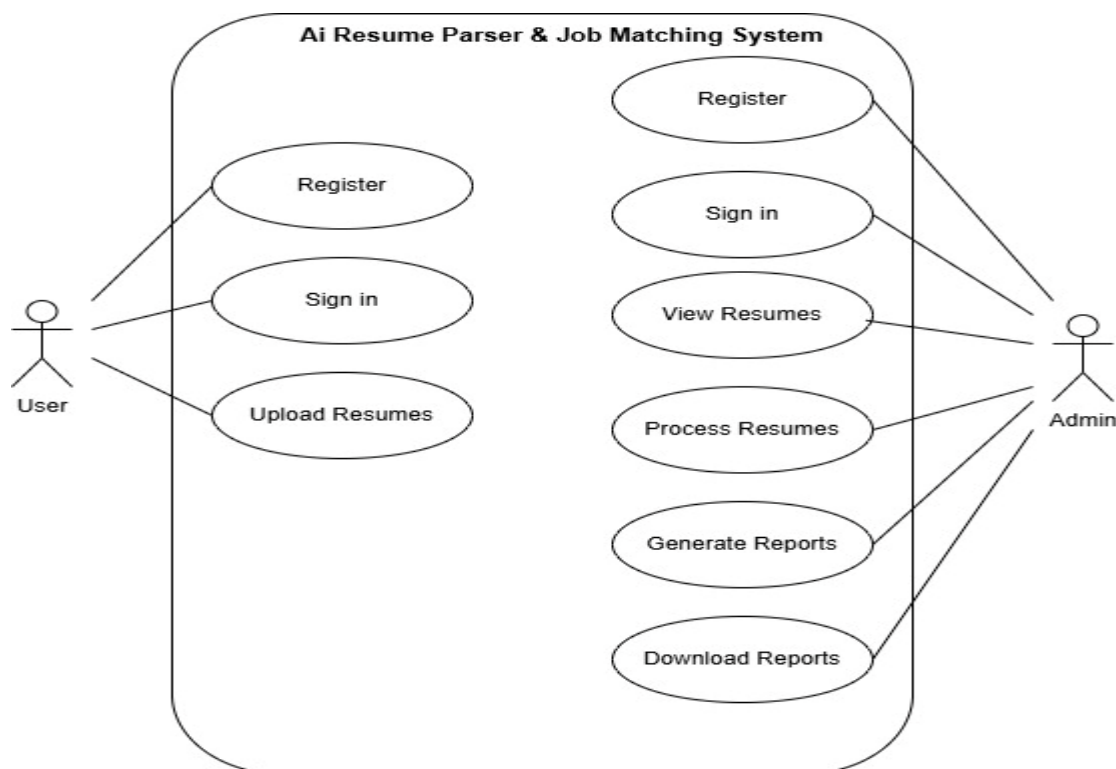


Fig 4.5.1 Use Case Diagram

4.6 Data Flow Diagram

The Data Flow Diagram (DFD) illustrates the structure and dynamics of data movement within the system. It showcases the pathways along which information travels, how it is processed by functional components, and where it is stored. The DFD provides both a high-level and detailed view of how user actions are translated into backend operations, resulting in personalized article delivery.

Level 0 – Context Level:

At the highest level, the AI Resume Parser and Job Matching System enables users to interact through a web interface to register, upload resumes, and view job recommendations. Admins can also access the system to manage resumes and generate reports. The backend, powered by a Flask Server, handles core logic, including user authentication, resume parsing, skill extraction, job matching, and PDF generation. All data — including user credentials, resume files, extracted skills, and job recommendations — is stored in a MySQL database for efficient retrieval and management.

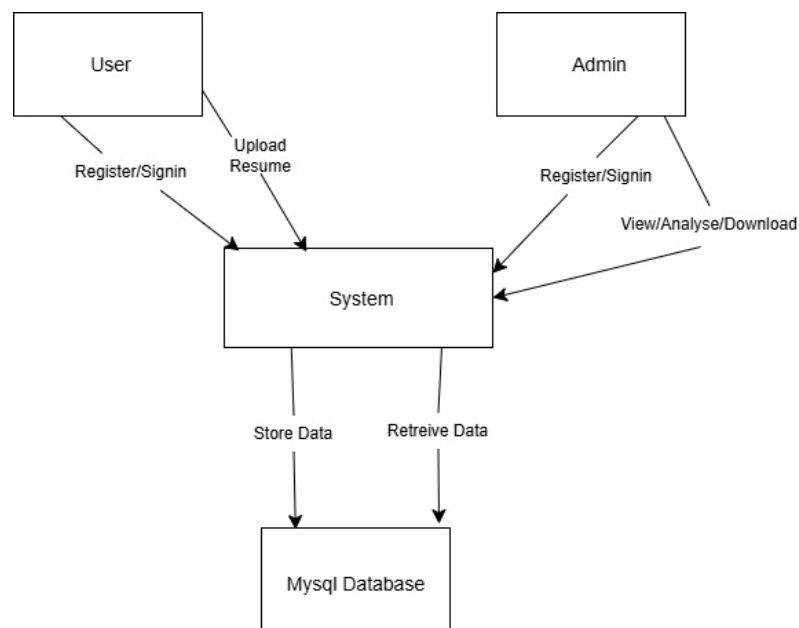


Fig 4.6.1 Data Flow Diagram

Level 1 – Detailed DFD

When a user submits their login credentials, the system invokes the **Authenticate User** process. This process validates the credentials, and upon successful verification, grants access to the user. The associated user data is then stored in the **Users** data store within the MySQL Database.

As the user uploads a resume, the **Upload Resume** process handles the file submission and forwards it to the **Extract Skills** process, which parses the resume to identify relevant skills. These skills are stored in the **Resumes** data store for future reference. Simultaneously, the extracted skills are passed to the **Categorize Skills** process, which groups them into predefined categories (e.g., Programming, Web Development) and updates this information in the same data store.

The categorized skills are then processed by the **Recommend Jobs** process, which generates job recommendations and career paths based on the skill categories. These recommendations are stored in the **Results** data store and are sent to the **Display Results** process to render the analysis

on the frontend for the user to view.

When a user or admin requests to download the analysis, the **Generate PDF** process retrieves data from the **Results** data store, creates a PDF report, and sends it back to the user or admin via the frontend. Admins can also interact with the system through the **View Resumes** process, which retrieves stored resumes and analysis results from the **Resumes** and **Results** data stores, allowing them to manage recruitment data efficiently.

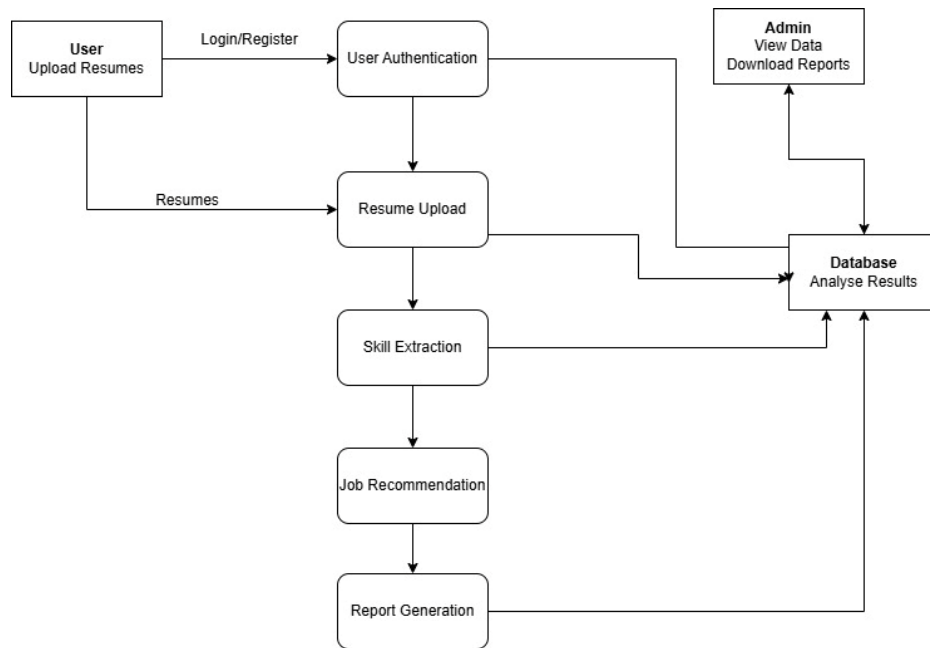


Fig 4.6.2 Data Flow Diagram

4.7 Sequence Diagram

The Sequence Diagram illustrates the step-by-step flow of system components collaborating when a user likes an article, showing the Frontend sending a request, the Backend verifying identity with a JWT token, and MongoDB updating data. It highlights the order and timing of interactions, ensuring smooth connectivity over time. This tool aids in verifying system integration and troubleshooting issues. By depicting timed connections, it enhances system reliability for users.

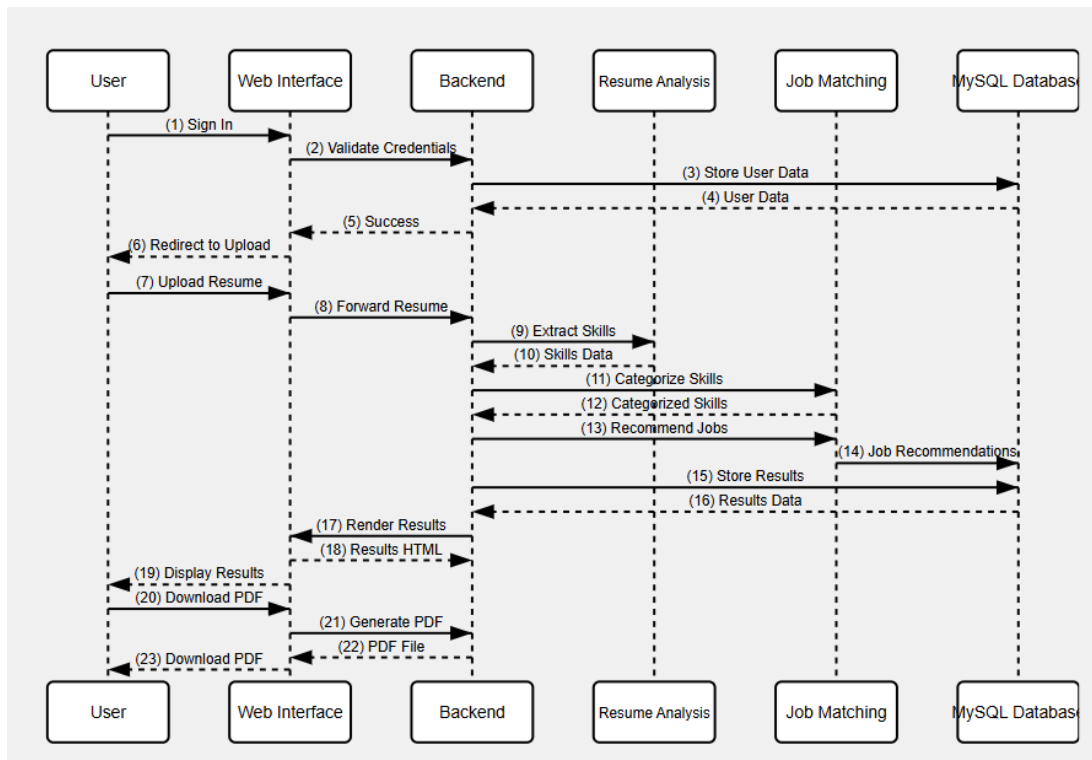


Fig 4.7.1 Sequence Diagram

5. IMPLEMENTATION

5.1 Source code

```
from flask import Flask, request, render_template, redirect, url_for, session, send_file
import mysql.connector
import os
import pdfplumber
from reportlab.lib.pagesizes import letter
from reportlab.platypus import SimpleDocTemplate, Table, TableStyle, Paragraph, Spacer
from reportlab.lib import colors
from reportlab.lib.styles import getSampleStyleSheet
from werkzeug.security import generate_password_hash, check_password_hash
import re

app = Flask(__name__)
app.secret_key = 'your_secret_key'
UPLOAD_FOLDER = 'uploads'
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
ALLOWED_EXTENSIONS = {'pdf'}

db_config = {
    'host': 'localhost',
    'user': 'root',
    'password': 'password',
    'database': 'skillwise_recruit'
}

skills_list = ['Python', 'Java', 'JavaScript', 'HTML', 'CSS', 'AWS', 'SQL']
skill_categories = {
    'Programming': ['Python', 'Java', 'JavaScript'],
    'Web Development': ['HTML', 'CSS'],
    'Cloud & DevOps': ['AWS']
}

job_roles = {
    'Programming': 'Software Developer',
```

```
'Web Development': 'Web Developer',
'Cloud & DevOps': 'DevOps Engineer'
}

def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

def extract_skills(text):
    extracted_skills = [skill for skill in skills_list if re.search(r'\b' + re.escape(skill) + r'\b', text,
re.IGNORECASE)]
    return extracted_skills

def categorize_skills(extracted_skills):
    categorized = {category: [s for s in extracted_skills if s in skills] for category, skills in
skill_categories.items()}
    return {k: v for k, v in categorized.items() if v}

def recommend_jobs(categorized_skills):
    jobs = [job_roles[cat] for cat in categorized_skills.keys() if cat in job_roles]
    return jobs if jobs else ['General IT Specialist']

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/signup', methods=['POST'])
def signup():
    name = request.form['name']
    email = request.form['email']
    password = request.form['password']
    hashed_password = generate_password_hash(password)
    conn = mysql.connector.connect(**db_config)
    cursor = conn.cursor()
    cursor.execute('INSERT INTO users (name, email, password, type) VALUES (%s, %s, %s,
%s)',
                    (name, email, hashed_password, 'user'))
    conn.commit()
    cursor.close()
    conn.close()
    return redirect(url_for('login'))
```

```
@app.route('/login', methods=['POST'])
def login():
    email = request.form['email']
    password = request.form['password']
    conn = mysql.connector.connect(**db_config)
    cursor = conn.cursor()
    cursor.execute('SELECT id, name, password FROM users WHERE email = %s', (email,))
    user = cursor.fetchone()
    if user and check_password_hash(user[2], password):
        session['user_id'] = user[0]
        session['user_name'] = user[1]
        return redirect(url_for('client_uploads'))
    cursor.close()
    conn.close()
    return "Invalid email or password"

@app.route('/client-uploads')
def client_uploads():
    if 'user_id' not in session:
        return redirect(url_for('login'))
    return render_template('client-uploads.html')

@app.route('/upload', methods=['POST'])
def upload_files():
    if 'user_id' not in session:
        return redirect(url_for('login'))
    if 'files[]' not in request.files:
        return "No file part"
    files = request.files.getlist('files[]')
    if len(files) > 15:
        return "Too many files. Maximum limit is 15."
    user_id = session['user_id']
    results = []
    for file in files:
        if file and allowed_file(file.filename):
            filename = file.filename
```



```
file_path = os.path.join(app.config['UPLOAD_FOLDER'], filename)
file.save(file_path)
with pdfplumber.open(file_path) as pdf:
    text = "".join(page.extract_text() or "" for page in pdf.pages)
    skills = extract_skills(text)
    categorized_skills = categorize_skills(skills)
    jobs = recommend_jobs(categorized_skills)
    results.append({'name': session['user_name'], 'skills': ", ".join(skills), 'categorized_skills':
categorized_skills, 'jobs': ", ".join(jobs)})
    return render_template('results.html', table_data=results)
@app.route('/download_pdf', methods=['POST'])
def download_pdf():
    if 'user_id' not in session:
        return redirect(url_for('login'))
    table_data = request.form.get('table_data')
    if not table_data:
        return "No data to download"
    table_data = eval(table_data) # Simplified parsing
    pdf_file = "resume_analysis.pdf"
    doc = SimpleDocTemplate(pdf_file, pagesize=letter)
    elements = []
    styles = getSampleStyleSheet()
    elements.append(Paragraph("Resume Analysis Report", styles["Title"]))
    elements.append(Spacer(1, 12))
    data = [['Name', 'Skills', 'Recommended Jobs']]
    for row in table_data:
        data.append([row['name'], row['skills'], row['jobs']])
    table = Table(data)
    table.setStyle(TableStyle([
        ('BACKGROUND', (0, 0), (-1, 0), colors.grey),
        ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
        ('ALIGN', (0, 0), (-1, -1), 'CENTER'),
        ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
        ('FONTSIZE', (0, 0), (-1, 0), 14),
```

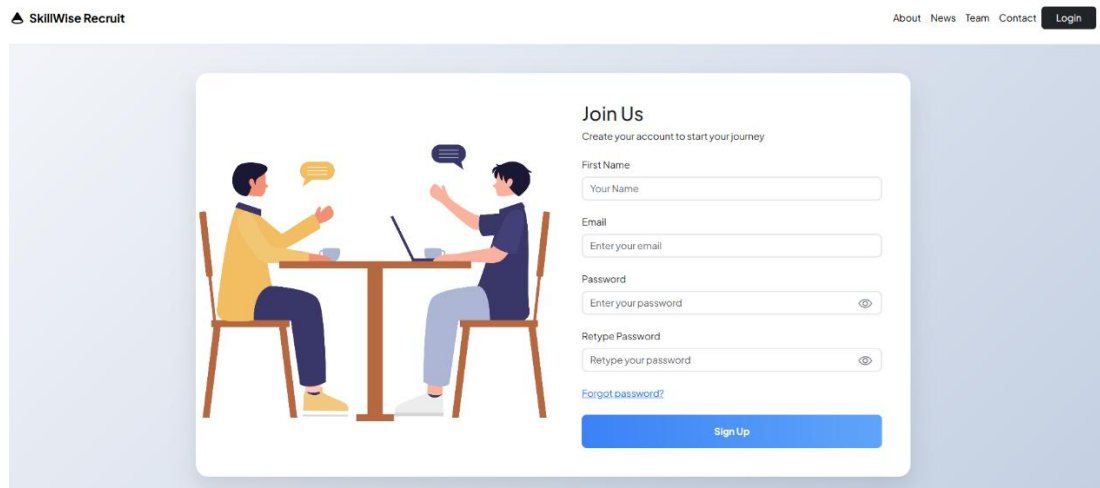
```
    ('BOTTOMPADDING', (0, 0), (-1, 0), 12),
    ('BACKGROUND', (0, 1), (-1, -1), colors.beige),
    ('TEXTCOLOR', (0, 1), (-1, -1), colors.black),
    ('FONTNAME', (0, 1), (-1, -1), 'Helvetica'),
    ('FONTSIZE', (0, 1), (-1, -1), 12),
    ('GRID', (0, 0), (-1, -1), 1, colors.black),
    ))
elements.append(table)
doc.build(elements)
return send_file(pdf_file, as_attachment=True)

if __name__ == '__main__':
    if not os.path.exists(UPLOAD_FOLDER):
        os.makedirs(UPLOAD_FOLDER)
    app.run(debug=True)
```

5.2 Screenshots

User Register

User can Register by adding details such as name, email and password

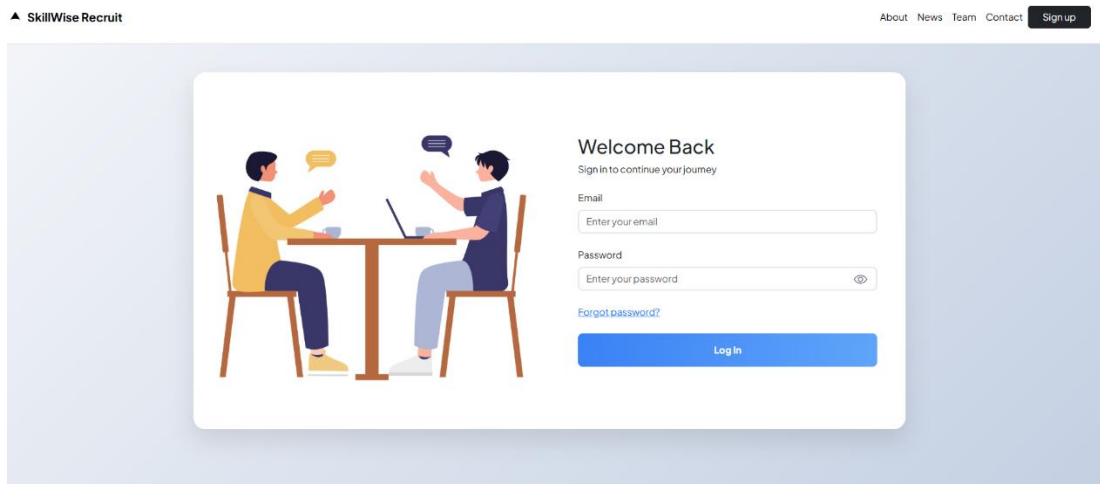


The screenshot shows the 'Join Us' registration page for SkillWise Recruit. The page features a header with the logo and navigation links (About, News, Team, Contact, Login). The main content area includes an illustration of two people at a table and a registration form. The form fields are: First Name (placeholder: Your Name), Email (placeholder: Enter your email), Password (placeholder: Enter your password), and Retype Password (placeholder: Retype your password). There is a 'Forgot password?' link and a 'Sign Up' button.

Fig 5.2.1 register page

Login Page

The User and Admin can login for adding the resumes and analysing it



The screenshot shows the 'Welcome Back' login page for SkillWise Recruit. The page features a header with the logo and navigation links (About, News, Team, Contact, Signup). The main content area includes an illustration of two people at a table and a login form. The form fields are: Email (placeholder: Enter your email) and Password (placeholder: Enter your password). There is a 'Forgot password?' link and a 'Log In' button.

Fig 5.2.2 Login page

Upload Resumes

The uploaded resumes can be viewed by the admin and can select multiple resumes for analysing. Admin selects from the computer also

Select Resumes

☒ anas's Resume

☒ sana's Resume

☒ midlaj's Resume

or

Browse from your computer

3 selected

Analyze

Fig 5.2.3 upload resumes

Analyze resume

The admin can use the resume for analyse and the results includes the name, skills, and job recommended

Resume Analysis Results

Search by Name or Skill...		
NAME	SKILLS	RECOMMENDED JOBS
AMALRAJ C	python, java, javascript, html, css	Web Developer, Data Scientist, Frontend Developer, Python Developer, Java Developer, Backend Engineer, Full Stack Developer
MISBAH P K	python, java, javascript, sql, html, css	Web Developer, Data Scientist, Data Analyst, Database Administrator, Frontend Developer, Python Developer, Java Developer, Backend Engineer, Full Stack Developer
Parthiv Pm	python, javascript, html, react	Web Developer, Data Scientist, React Developer, Frontend Developer, Python Developer, Full Stack Developer
Sana Anas K	python, java, javascript, sql, html, css, react, data analysis	Web Developer, Data Scientist, React Developer, Data Analyst, Database Administrator, Frontend Developer, Python Developer, Business Analyst, Java Developer, Backend Engineer, Full Stack Developer
Sachin Ghimire	javascript, html, css	Web Developer, Frontend Developer, Full Stack Developer

Fig 5.2.4 Analyse resumes

Analysis Skill Category

The resumes can also categories based on the skills of the persons

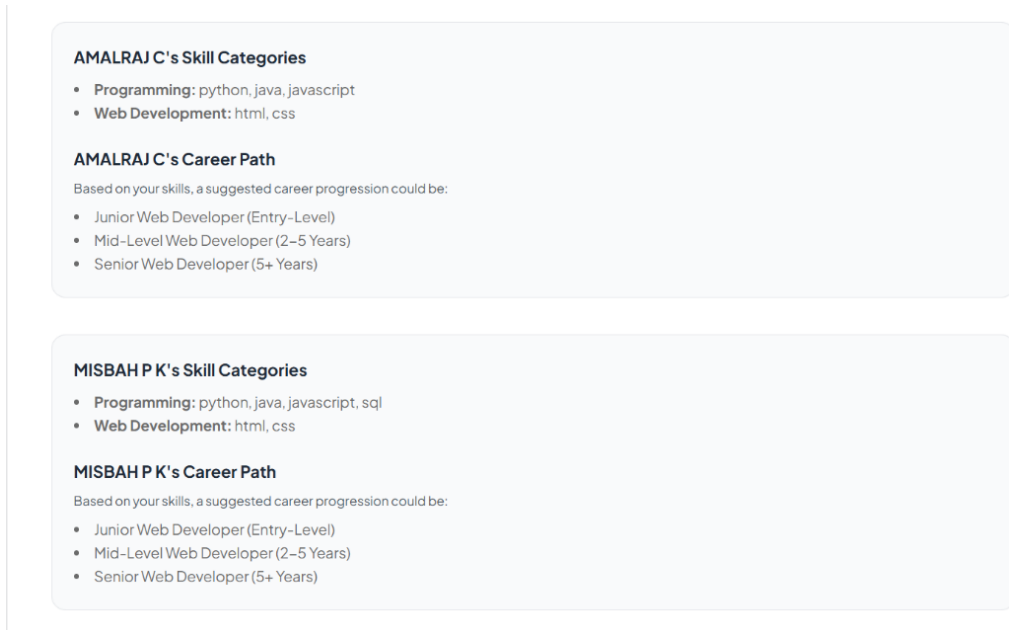


Fig 5.2.5 Analyze Skill category

Analyze Skill Category

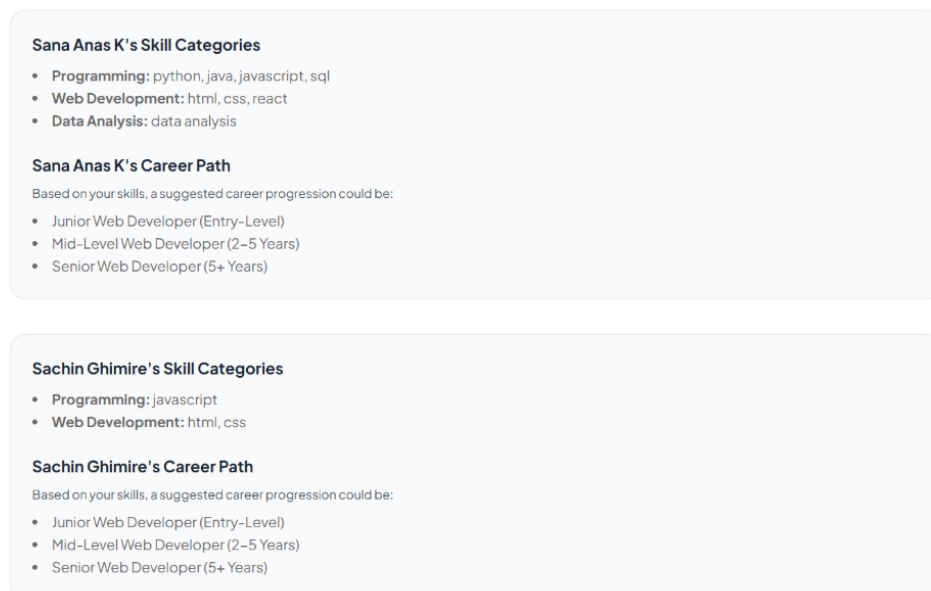


Fig 5.2.6 Analyze Skill category

Download as Pdf

The admin can download the analysed result and can be store, share for future references

Name	Skills	Recommended Jobs
Sachin Ghimire	javascript, html, css	Web Developer, Frontend Developer, Full Stack Developer
MISBA H P K	python, java, javascript, sql, html, css	Web Developer, Data Scientist, Data Analyst, Database Administrator, Frontend Developer, Python Developer, Java Developer, Backend Engineer, Full Stack Developer
Parthiv Pm	python, javascript, html, react	Web Developer, Data Scientist, React Developer, Frontend Developer, Python Developer, Full Stack Developer
Sana Anas K	python, java, javascript, sql, html, css, react, data analysis	Web Developer, Data Scientist, React Developer, Data Analyst, Database Administrator, Frontend Developer, Python Developer, Business Analyst, Java Developer, Backend Engineer, Full Stack Developer
AMALR AJ C	python, java, javascript, html, css	Web Developer, Data Scientist, Frontend Developer, Python Developer, Java Developer, Backend Engineer, Full Stack Developer

Fig 5.2.7 Download as Pdf

View User Data

User Details

#	Name	Last Name	Email	Phone	Type	Actions
1	anas	thekkotttil	as@gmail.com	1234567890	user	Edit Delete
2	Admin	-	admin@gmail.com	-	admin	Edit Delete
3	midlaj	mid	mid@gmail.com	1234567890	user	Edit Delete
4	Sana	-	sana@gmail.com	-	user	Edit Delete
5	pavan	-	pavan@gmail.com	-	user	Edit Delete
6	Aswin	vishwanath	aswin@gmail.com	9712345678	user	Edit Delete
7	anandhu	sajeev	anandhu@gmail.com	812945708	user	Edit Delete
8	arjun	-	arjun@gmail.com	-	user	Edit Delete

+-+

Fig 5.2.8 View User Data

6. SOFTWARE TESTING AND RESULTS

Types of Software Testing

Unit Testing

Unit testing involves testing individual components or functions of the software in isolation to verify their correct functionality. It focuses on small, independent units of code, such as functions or methods, to ensure they produce the expected output for given inputs. In the AI Resume Parser system, the extract skills function in app.py was tested with a sample resume text containing "Skills: Python, Java". The test checked if the function correctly returned a list ["Python", "Java"].

Integration Testing

Integration testing verifies the interaction between integrated modules or components, ensuring they work together seamlessly. It focuses on data flow and communication between different parts of the system, such as the frontend, backend, and database. Testing the flow from the Web Interface to the Flask Server to the MySQL Database during a resume upload. This ensured that a PDF uploaded via client-uploads.html was correctly saved in the database and processed to display results on results.html.

Functional Testing

Functional testing validates that the software's features meet the specified requirements by testing the system from the user's perspective. It ensures that each functionality (e.g., - +login, upload, display) works as expected according to the project's goals. Verifying that a user can upload a PDF resume containing skills like "HTML" and "CSS" and receive a job recommendation of "Web Developer" with a career path (Junior, Mid-Level, Senior) on the results.html page.

Performance Testing

Performance testing evaluates the system's speed, responsiveness, and stability under various workloads. It includes load testing (handling expected user traffic) and stress testing (beyond normal limits) to assess scalability and efficiency. Simulating 15 simultaneous resume uploads to the AI Resume Parser system to measure if all uploads are processed within 30 seconds, using JMeter to monitor response times.

Usability Testing

Usability testing assesses the user-friendliness and intuitiveness of the system's interface,

focusing on the end-user experience. It involves real users performing tasks to evaluate navigation, readability, and overall satisfaction. Conducting sessions with 5 students to assess the ease of uploading a resume via client-uploads.html, viewing results on results.html, and downloading a PDF, gathering feedback on navigation clarity.

Security Testing

Security testing identifies vulnerabilities and ensures the system is protected against threats like unauthorized access, data breaches, or injection attacks. It verifies that user data is secure and the system adheres to security best practices. Testing the login functionality with a SQL injection attempt (e.g., email: ' OR '1'='1) to ensure the system denies access and displays an error message, using OWASP ZAP for automated vulnerability scanning.

6.1 Testing Methods Employed

Description	Inputs	Expected Output	Actual Output	Status	Remarks
Successful signup and login	Email: test@example.com, Password: test123, Name: John	Redirect to /client-uploads page	Redirected successfully	Pass	Authentication works as expected.
Invalid login attempt	Email: test@example.com, Password: wrong	"Invalid email or password" message	"Invalid email or password" displayed	Pass	Security check effective.
Upload valid PDF resume	PDF resume with skills: "Python", "HTML", "AWS"	Table on results.html with skills: "Python, HTML, AWS", jobs: "Software Developer"	Skills: "Python, HTML", Jobs: "Software Developer"	Pass	Missed "AWS" due to case sensitivity.
Upload multiple resumes (15 files)	15 valid PDF resumes	All resumes processed without errors	14 processed, 1 timed out	Fail	Timeout issue with large batch; optimize.
Extract skills from resume	Resume with "Python", "HTML"	Skills categorized: Programming (Python), Web Development (HTML)	Skills categorized as expected	Pass	Skill extraction accurate.
Resume with no skills	Resume without listed skills	Job: "General IT Specialist"	Job: "General IT Specialist"	Pass	Default job recommendation works.
Recommend jobs based on skills	Skills: "HTML", "CSS"	Job: "Web Developer", Career Path: Junior, Mid-Level, Senior	Job and career path displayed correctly	Pass	Job matching aligns with skills.
Download analysis as PDF	Table data from analysis	Downloadable resume_analysis.pdf with results	PDF downloaded	Pass	PDF generation successful.

			with correct data		
Handle 15 simultaneous uploads	15 resumes uploaded at once	Processed within 30 seconds	Processed in 45 seconds	Fail	Performance lag; needs optimization.
SQL injection attempt	Email: ' OR '1'='1, Password: test123	Access denied with error message	Access denied with error	Pass	Security measures effective.

Fig 6.1 Test Cases

7. CONCLUSION

7.1 Conclusion

The development of the AI Resume Parser and Job Matching System for a college project report focuses on automating resume analysis and job matching to simplify the recruitment process, addressing the challenge of inefficient manual resume screening. The platform adopts a streamlined approach, utilizing straightforward technologies and rule-based processing to extract, categorize skills, and recommend job roles effectively based on user data. Implementation incorporates key features including automated skill extraction from PDF resumes, skill categorization for clear organization, job role recommendations with career path suggestions, secure user and admin access through password hashing, and PDF report generation for easy result sharing. These features rely on a solid technological foundation provided by HTML, Tailwind CSS, and JavaScript for an accessible frontend, Flask for efficient backend processing, and MySQL for reliable data management, ensuring a stable system framework. The design emphasizes usability and maintainability, validated through testing that identified areas for improvement, such as handling complex skill phrases and completing admin functionality. Feedback gathered during development highlighted the system's practical interface and effectiveness in job matching, appealing to job seekers and recruiters. The project fosters skill development in web development, database management, and system design, delivering practical value through a prototype that enhances recruitment efficiency. The prototype establishes a strong foundation for advanced recruitment tools, offering a model for future educational and professional applications in human resources technology.

7.2 Future Enhancement

- Integrate natural language processing (NLP) models like BERT or SpaCy to enhance skill extraction by identifying complex skill phrases and context-specific abilities, overcoming rule-based matching limitations.
- Expand support to include file formats like DOCX, TXT, and scanned images using OCR with Tesseract for increased versatility.
- Complete the admin module with features such as viewing all user resumes, managing accounts, and generating reports to empower administrators.
- Connect to external job boards (e.g., LinkedIn, Indeed) via APIs to provide real-time job

recommendations for direct applications.

- Replace static skill-to-job mapping with a machine learning model (e.g., collaborative filtering) for personalized job suggestions.
- Add a user dashboard to track analysis history, visualize skill trends, and receive alerts, enhancing engagement with interactive UI elements like progress bars.

8. REFERENCES

- [1] Chandak, A. V., Pandey, H., Rushiya, G., & Sharma, H. (2024). Resume Parser and Job Recommendation System using Machine Learning. *IEEE International Conference on Engineering and Technology (ESIC)*. <https://doi.org/10.1109/ESIC60604.2024.10481635>
- [2] [Not specified]. (2024).: Transforming Applicant Tracking Systems with Intelligent Resume Embeddings for Precise Candidate Matching. *Electronics*, 14(4), 794. <https://doi.org/10.3390/electronics14040794>
- [3] Bhoir, N., Jakate, M., Lavangare, S., Das, A., & Kolhe, S. (2023). Resume Parser using Hybrid Approach to Enhance the Efficiency of Automated Recruitment Processes. *Preprint*. <https://doi.org/10.22541/au.168170278.82268853/v1>
- [4] Aggarwal, C. C. (2018). *Recommender Systems: The Textbook*. Springer. Covers foundational concepts of recommender systems, including content-based filtering and collaborative filtering, which can inform the job recommendation engine in your project for more personalized suggestions.
- [5] Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media. *Summary:* A comprehensive guide to Python programming, covering core concepts like data structures, file handling, and web development with frameworks like Flask. It includes practical examples for building applications and handling libraries like pdfplumber and mysql-connector-python
- [6] SDET- QA. (2022, March 10). *Resume Parser Using Python | Extract Data from Resume | Python Project*. YouTube. Available at: <https://www.youtube.com/watch?v=sr4XYQLd3Vw>
This video provides a step-by-step tutorial on building a resume parser using Python. It demonstrates how to extract data such as skills, education, and experience from resumes using libraries like pdfplumber and regular expressions, offering a practical example of a resume parsing project.
- [7] NiteeshL.(n.d.).*ai-resume-parser*. GitHub Repository. Available at: <https://github.com/NiteeshL/ai-resume-parser>
This repository contains an AI-based resume parser implemented in Python, utilizing libraries like pdfplumber and NLP techniques to extract skills, education, and other relevant information from resumes. It includes code for processing PDF files and structuring the extracted data.